

PROJECTE FINAL

DESENVOLUPAMENT EN WEB ENTORN SERVIDOR

EL MEU "PAS A PAS"

CREACIÓ API REST SEGURA AMB LARAVEL

Sana Rut Sabir París
2n DAW
10/05/2026

1) Creació aplicació:

composer create-project laravel/laravel api-tasques

```
INFO  Preparing database.
Creating migration table ..... 15.08ms DONE
INFO  Running migrations.
0001_01_01_000000_create_users_table ..... 32.13ms DONE
0001_01_01_000001_create_cache_table ..... 21.89ms DONE
0001_01_01_000002_create_jobs_table ..... 27.05ms DONE
```

cd api-tasques

php artisan serve

2) Configurar base de dades

CREATE DATABASE api_tasques;

.env:

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=api_tasques

DB_USERNAME=root

DB_PASSWORD=

3) Crear model i migració de Tasca

php artisan make:model Task -m

```
INFO  Model [C:\Users\HP\api-tasques\app\Models\Task.php] created successfully.
INFO  Migration [C:\Users\HP\api-tasques\database\Migrations\2026_05_05_193424_create_tasks_table.php] created successfully.
```

4) Editar MIGRACIÓ: database>Migrations>create_task_table...

```
return new class extends Migration {
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->id();

            // Cada tasca pertany a un usuari concret.
            $table->foreignId('user_id')->constrained()->onDelete('cascade');

            // Dades principals de la tasca.
            $table->string('nom');
            $table->text('comentari');
            $table->string('categoria');

            $table->timestamps();
        });
    }
}
```

Faig la migració:

```
C:\Users\HP\api-tasques>php artisan migrate

INFO Running migrations.

2026_05_05_193424_create_tasks_table ..... 32.12ms DONE
```

5) Edició MODEL Task:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

0 references | 0 implementations
class Task extends Model
{
    // Camps que Laravel permet omplir des del controlador.
    0 references
    protected $fillable = [
        'user_id',
        'nom',
        'comentari',
        'categoria'
    ];
}
```

6) Relació User-Task:

app/Models/User.php:

```
use App\Models\Task;

// Un usuari té moltes tasques.
0 references | 0 overrides
public function tasks()
{
    return $this->hasMany(Task::class);
}
```

app/Models/Task.php:

```
use App\Models\User;

// Cada tasca pertany a un usuari.
0 references | 0 overrides
public function user()
{
    return $this->belongsTo(User::class);
}
```

Abans de començar amb l'autenticació amb Token:

composer require laravel/sanctum

Ara l'instal·lo:

php artisan sanctum:install

```
81 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force
[INFO] No publishable resources for tag [laravel-assets].
No security vulnerability advisories found.
Using version ^4.3 for laravel/sanctum
C:\Users\HP\api-tasques>php artisan sanctum:install
[ERROR] Command "sanctum:install" is not defined. Did you mean one of these?
  migrate:install
  sail:install
  sanctum:prune-expired
C:\Users\HP\api-tasques>
```

... 🤔 🤔

Doncs ho torno a intentar (segons el meu amic *Chatgpt* això es pot deure a la versió del Laravel)

php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"

```
C:\Users\HP\api-tasques>php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
[INFO] Publishing assets.
Copying directory [C:\Users\HP\api-tasques\vendor\laravel\sanctum\database\Migrations] to [C:\Users\HP\api-tasques\database\Migrations] DONE
Copying file [C:\Users\HP\api-tasques\vendor\laravel\sanctum\config\sanctum.php] to [C:\Users\HP\api-tasques\config\sanctum.php] DONE
```

Ara:

php artisan migrate

```
C:\Users\HP\api-tasques>php artisan migrate
[INFO] Running migrations.
2026_05_05_200930_create_personal_access_tokens_table ..... 204.52ms DONE
```

Explicació:

Sanctum és un paquet extern que he instal·lat, però per defecte:

- ve “amagat” dins de vendor
- no modifica directament el projecte

Amb vendor:publish:

→ porto aquests fitxers a projecte perquè els pugui utilitzar i modificar

Què aporta això?

Principalment:

- crea la configuració de Sanctum
- prepara la migració de la taula personal_access_tokens
- permet que Laravel pugui generar i validar tokens

Dit això, segueixo amb les modificacions :)

- Preparar el model User per Sanctum

app/Models/User.php

```
use Laravel\Sanctum\HasApiTokens;
```

Canviar línia → use HasFactory, Notifiable;

```
use HasApiTokens, HasFactory, Notifiable;
```

Això permet que el model User pugui crear tokens amb:

```
$user->createToken('api-token')->plainTextToken;
```

- **Crear controlador d'autenticació**

php artisan make:controller Api/AuthController

```
C:\Users\HP\api-tasques>php artisan make:controller Api/AuthController
INFO Controller [C:\Users\HP\api-tasques\app\Http\Controllers\Api\AuthController.php] created successfully.
```

- **Preparar AuthController**

<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;

use Illuminate\Http\Request;

use App\Models\User;

use Illuminate\Support\Facades\Hash;

class AuthController extends Controller

{

// Registre d'usuari

public function register(Request \$request)

{

// Validació de dades

\$request->validate([

'name' => 'required|string',

'email' => 'required|email|unique:users',

'password' => 'required|min:6'

]);

// Crear usuari

\$user = User::create([

'name' => \$request->name,

'email' => \$request->email,

'password' => Hash::make(\$request->password)

]);

// Retorn JSON

return response()->json([

'success' => true,

'message' => 'Usuari registrat correctament',

'data' => \$user

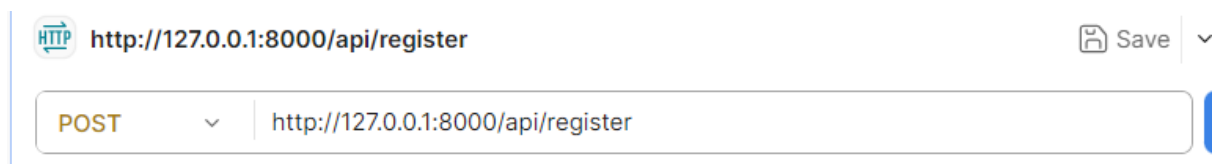
], 201); }}

- Crear ruta /web/register

```
api-tasques > routes > web.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\Api\AuthController;
5
6  Route::get('/', function () {
7      return view('welcome');
8  });
9
10 // Ruta API registre
11 Route::post('/api/register', [AuthController::class, 'register']);
```

Ir a POSTMAN

http://127.0.0.1:8000/api/register



Escribir a BODY:



Send:

The screenshot shows a web client interface with a top navigation bar containing 'Docs', 'Params', 'Authorization', 'Headers 9', 'Body', 'Scripts', 'Tests', and 'Settings'. The 'Body' tab is selected, showing a JSON object:

```
{  "name": "Rut",  "email": "rut@test.com",  "password": "123456"}
```

. Below the JSON, there's a status bar indicating '419 unknown status', '4.35 s', and '7.19 KB'. Underneath, there's a preview section with a dropdown set to 'HTML', a 'Preview' button, and a 'Debug with AI' button. The HTML preview shows the following code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Page Expired</title>
    <style>
      /*! normalize.css v8.0.1 | MIT License | github.com/necolas/normalize.
      css */html{line-height:1.15;-webkit-text-size-adjust:100%}body
      {margin:0;a{background-color:transparent}code{font-family:monospace,
```



Doncs torno a intentar-ho.

Torno a deixar web.php neta:

```
api-tasques > routes > web.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4
5  Route::get('/', function () {
6      return view('welcome');
7  });
8
```


Creo *api.php*:

```
api-tasques > routes > api.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\Api\AuthController;
5
6  Route::post('/register', [AuthController::class, 'register']);
```

A *bootstrap/app.php*, afegiria això:

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
```

Torno a provar POSTMAN:

The screenshot shows the Postman interface. At the top, the URL is `http://127.0.0.1:8000/api/register`. The method is set to **POST**. The request body is in **JSON** format and contains the following data:

```
{
  "name": "Rut",
  "email": "rut@test.com",
  "password": "123456"
}
```

The response status is **201 Created**. The response body is in **JSON** format and contains the following data:

```
{
  "success": true,
  "message": "Usuari registrat correctament",
  "data": {
    "name": "Rut",
    "email": "rut@test.com",
    "updated_at": "2026-05-05T20:57:10.000000Z",
    "created_at": "2026-05-05T20:57:10.000000Z",
    "id": 1
  }
}
```

REGISTRE OK!!! ✓

Ara anem amb LOGIN...

app/Http/Controllers/Api/AuthController.php

```
// Login d'usuari
0 references | 0 overrides
public function login(Request $request)
{
    // Validem que arribin email i password.
    $request->validate([
        'email' => 'required|email',
        'password' => 'required'
    ]);

    // Busquem l'usuari pel seu email.
    $user = User::where('email', $request->email)->first();

    // Comprovem si l'usuari existeix i si la contrasenya és correcta.
    if (!$user || !Hash::check($request->password, $user->password)) {
        return response()->json([
            'success' => false,
            'message' => 'Credencials incorrectes'
        ], 401);
    }

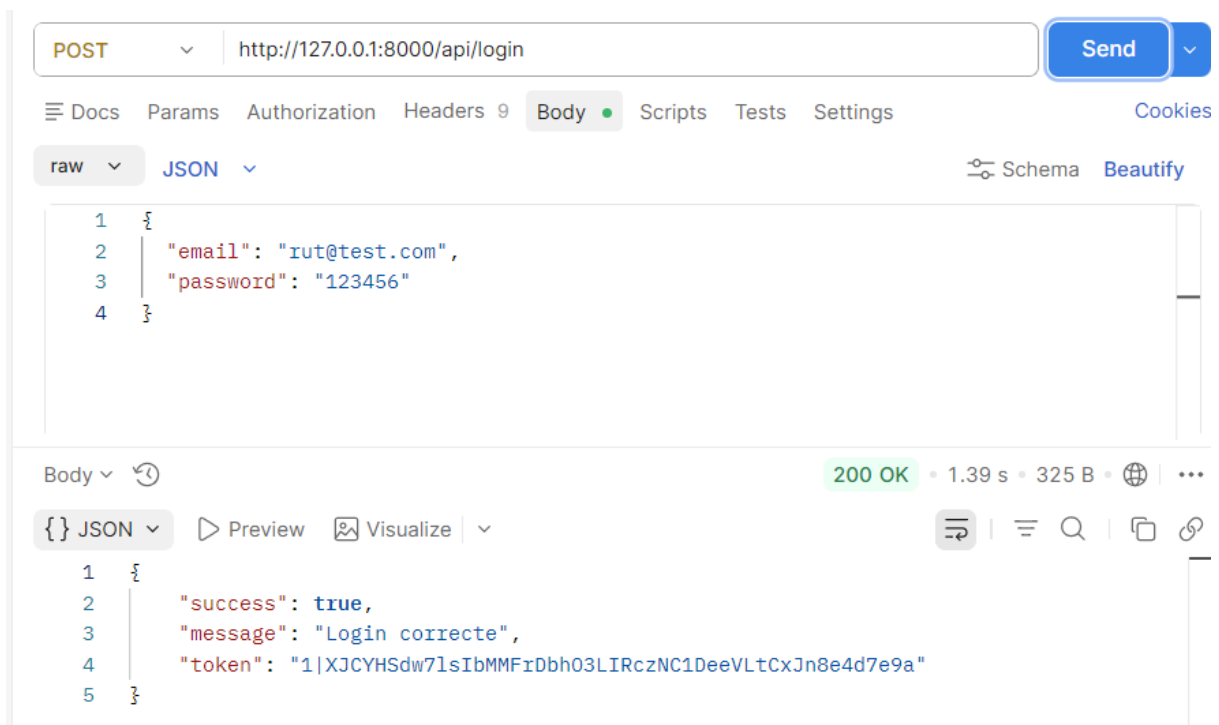
    // Creem el token d'accés amb Sanctum.
    $token = $user->createToken('api-token')->plainTextToken;

    // Retornem el token en format JSON.
    return response()->json([
        'success' => true,
        'message' => 'Login correcta',
        'token' => $token
    ], 200);
}
```

Ara, a *routes/api.php*

```
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);
```

A POSTMAN—> LOGIN



LOGIN OK ✓

Ara vaig a crear el Controller de Tasques:

php artisan make:controller Api/TaskController --api

```
C:\Users\HP\api-tasques>php artisan make:controller Api/TaskController --api
INFO Controller [C:\Users\HP\api-tasques\app\Http\Controllers\Api\TaskController.php] created successfully.
```

Preparo TaskController:

app/Http/Controllers/Api/TaskController.php

```

<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\Task;

0 references | 0 implementations
class TaskController extends Controller
{
    // GET /api/tasks
    // Llistar totes les tasques de l'usuari autenticat
    0 references | 0 overrides
    public function index(Request $request)
    {
        // Obtenim l'usuari autenticat a partir del token
        $user = $request->user();

        // Recuperem només les seves tasques
        $tasks = Task::where('user_id', $user->id)->get();

        return response()->json([
            'success' => true,
            'data' => $tasks
        ], 200);
    }
}

```

```

// POST /api/tasks
// Crear una nova tasca
0 references | 0 overrides
public function store(Request $request)
{
    // Validació
    $request->validate([
        'nom' => 'required',
        'comentari' => 'required',
        'categoria' => 'required'
    ]);

    $user = $request->user();

    // Crear tasca associada a l'usuari
    $task = Task::create([
        'user_id' => $user->id,
        'nom' => $request->nom,
        'comentari' => $request->comentari,
        'categoria' => $request->categoria
    ]);

    return response()->json([
        'success' => true,
        'message' => 'Tasca creada correctament',
        'data' => $task
    ], 201);
}

```

```
// GET /api/tasks/{id}
0 references | 0 overrides
public function show(Request $request, $id)
{
    $user = $request->user();

    $task = Task::where('id', $id)
        ->where('user_id', $user->id)
        ->first();

    if (!$task) {
        return response()->json([
            'success' => false,
            'message' => 'Tasca no trobada'
        ], 404);
    }

    return response()->json([
        'success' => true,
        'data' => $task
    ], 200);
}
```

```
// PUT /api/tasks/{id}
0 references | 0 overrides
public function update(Request $request, $id)
{
    $user = $request->user();

    $task = Task::where('id', $id)
        ->where('user_id', $user->id)
        ->first();

    if (!$task) {
        return response()->json([
            'success' => false,
            'message' => 'No tens permisos'
        ], 403);
    }

    $task->update($request->all());

    return response()->json([
        'success' => true,
        'message' => 'Tasca actualitzada',
        'data' => $task
    ], 200);
}
```

```
// DELETE /api/tasks/{id}
0 references | 0 overrides
public function destroy(Request $request, $id)
{
    $user = $request->user();

    $task = Task::where('id', $id)
        ->where('user_id', $user->id)
        ->first();

    if (!$task) {
        return response()->json([
            'success' => false,
            'message' => 'No tens permisos'
        ], 403);
    }

    $task->delete();

    return response()->json([
        'success' => true,
        'message' => 'Tasca eliminada'
    ], 200);
}
```

Amb això ja tinc el CRUD COMPLET i FILTRAT PER USUARI.

El següent serà protegir rutes amb Token:

routes/api.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\TaskController;

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

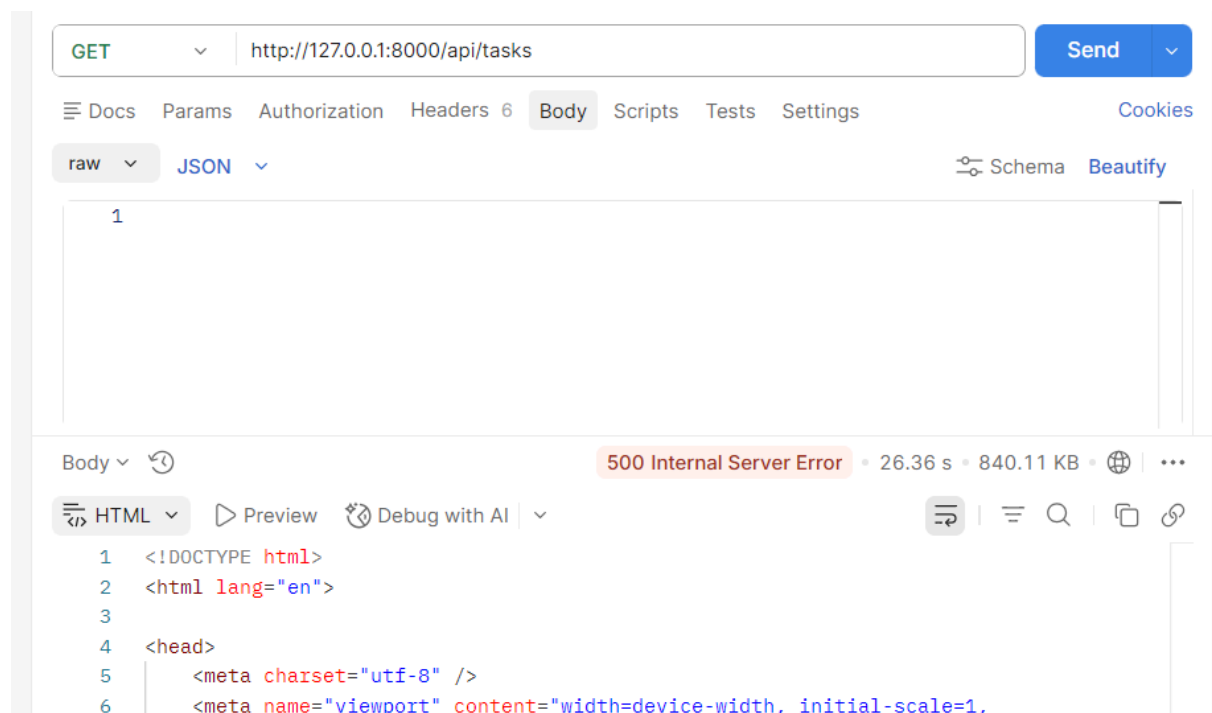
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/tasks', [TaskController::class, 'index']);
    Route::post('/tasks', [TaskController::class, 'store']);
    Route::get('/tasks/{id}', [TaskController::class, 'show']);
    Route::put('/tasks/{id}', [TaskController::class, 'update']);
    Route::delete('/tasks/{id}', [TaskController::class, 'destroy']);
});
```

Què fa això?

auth:sanctum = només entra qui tingui token.

- El pas que toca ara és fer GET (sense Token, per provar):

The image shows a REST client interface. At the top, the method is set to 'GET' and the URL is 'http://127.0.0.1:8000/api/tasks'. Below this, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', 'Tests', and 'Settings'. The 'Body' tab is currently selected. On the right side, there is a 'Send' button. Below the tabs, there is a 'raw' button and a 'JSON' button. The 'JSON' button is selected. At the bottom, there is a large text area for the response body, which currently contains the number '1'.



GET http://127.0.0.1:8000/api/tasks

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

1

Body 500 Internal Server Error • 26.36 s • 840.11 KB • ...

HTML Preview Debug with AI

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8" />
6   <meta name="viewport" content="width=device-width, initial-scale=1,
```

... 😊

Jo diria que això no és exactament el que ha de sortir. Cambio estratègia.

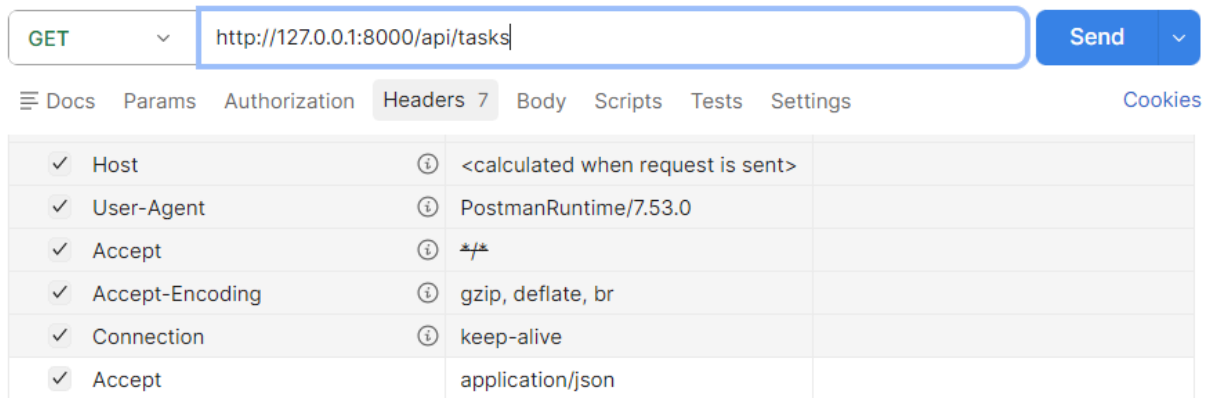
```
C:\Users\HP\api-tasques>php artisan route:list

GET|HEAD / ..... routes/web.php:5
POST api/login ..... Api\AuthController@login
POST api/register ..... Api\AuthController@register
GET|HEAD api/tasks ..... Api\TaskController@index
POST api/tasks ..... Api\TaskController@store
GET|HEAD api/tasks/{id} ..... Api\TaskController@show
PUT api/tasks/{id} ..... Api\TaskController@update
DELETE api/tasks/{id} ..... Api\TaskController@destroy
GET|HEAD sanctum/csrf-cookie ..... sanctum.csrf-cookie > Laravel\Sanctum > CsrfCookieController@show
GET|HEAD storage/{path} storage.local > vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServiceProvid...
PUT storage/{path} storage.local.upload > vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServic...
GET|HEAD up ..... vendor/laravel/framework/src/Illuminate/Foundation/Configuration/ApplicationBuilder.php:219

Showing [12] routes
```

Quan no hi ha Token, Laravel intenta respondre amb routes/web.php (per això html).

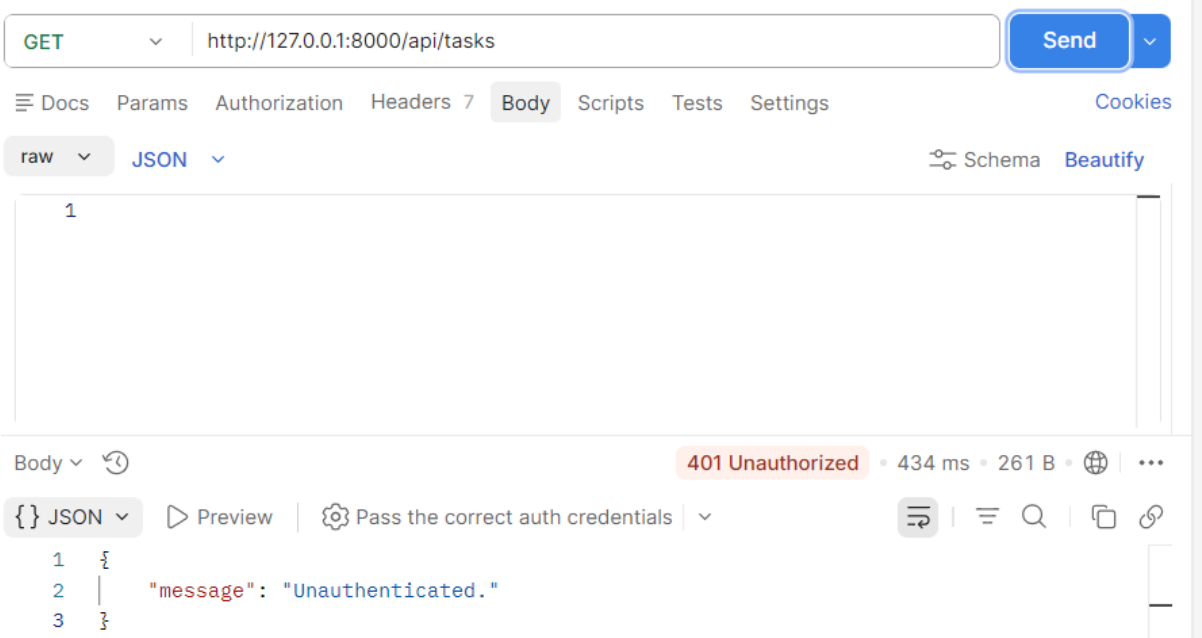
Per tal de solucionar-ho, afegeixo **Accept: application/json**



The image shows the 'Headers' tab in Postman. The URL is 'http://127.0.0.1:8000/api/tasks'. The 'Accept' header is set to 'application/json'. Other headers include Host, User-Agent, Accept-Encoding, and Connection.

Key	Value
Host	<calculated when request is sent>
User-Agent	PostmanRuntime/7.53.0
Accept	application/json
Accept-Encoding	gzip, deflate, br
Connection	keep-alive

Tornem-hi:



The image shows the 'Body' tab in Postman. The URL is 'http://127.0.0.1:8000/api/tasks'. The response is a 401 Unauthorized error. The JSON body is: { "message": "Unauthenticated." }.

```
1 {
2   "message": "Unauthenticated."
3 }
```

Ara NO em dona el format d'error que requereix l'exercici, però això ho polirè després.

El pas següent és testejar amb Token

Torno a modificar Header de POSTMAN

✓ Accept	application/json
✓ Authorization	Bearer 1 XJCYHSdw7IsIbMMFrDb...

Faig Send i...

The screenshot shows the Postman interface. At the top, a GET request is configured to `http://127.0.0.1:8000/api/tasks`. The Authorization header is set to `Bearer 1|XJCYHSdw7IsIbMMFrDb...`. The response status is `200 OK` with a time of `1.27 s` and a size of `247 B`. The response body is a JSON object:

```
{
  "success": true,
  "data": []
}
```

Aquest exemple sí és correcte.

Ara anem amb CREATE:

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:8000/api/tasks`. The request body is a JSON object with the following data:

```
{
  "nom": "Estudiar Laravel",
  "comentari": "Fer proves del CRUD amb Postman",
  "categoria": "Estudis"
}
```

The response is a 201 Created status, indicating the task was successfully created. The response body is a JSON object:

```
{
  "success": true,
  "message": "Tasca creada correctament",
  "data": {
    "user_id": 1,
    "nom": "Estudiar Laravel",
    "comentari": "Fer proves del CRUD amb Postman",
    "categoria": "Estudis",
    "updated_at": "2026-05-06T09:07:35.000000Z",
    "created_at": "2026-05-06T09:07:35.000000Z",
    "id": 1
  }
}
```

CREATE OK ✓

The screenshot shows a Postman interface for a GET request to `http://127.0.0.1:8000/api/tasks`. The response is a 200 OK status, indicating the tasks were successfully retrieved. The response body is a JSON object:

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "user_id": 1,
      "nom": "Estudiar Laravel",
      "comentari": "Fer proves del CRUD amb Postman",
      "categoria": "Estudis",
      "created_at": "2026-05-06T09:07:35.000000Z",
      "updated_at": "2026-05-06T09:07:35.000000Z"
    }
  ]
}
```

GET api/Task (READ) funciona ✓

GET /api/tasks/{id}

A screenshot of the Postman application interface. The top bar shows a GET request to `http://127.0.0.1:8000/api/tasks/1` with a 'Send' button. Below the bar, tabs for Docs, Params, Authorization, Headers, Body, Scripts, Tests, and Settings are visible. The 'Body' tab is selected, showing a 'raw' JSON response. The response status is '200 OK' with a response time of 866 ms and a size of 444 B. The JSON body is as follows:

```
1 {
2   "success": true,
3   "data": {
4     "id": 1,
5     "user_id": 1,
6     "nom": "Estudiar Laravel",
7     "comentari": "Fer proves del CRUD amb Postman",
8     "categoria": "Estudis",
9     "created_at": "2026-05-06T09:07:35.000000Z",
10    "updated_at": "2026-05-06T09:07:35.000000Z"
11  }
12 }
```

READ OK ✓

- Ara començaré amb proves de seguretat.

A screenshot of the Postman application interface. The top bar shows a POST request to `http://127.0.0.1:8000/api/register` with a 'Send' button. Below the bar, tabs for Docs, Params, Authorization, Headers, Body, Scripts, Tests, and Settings are visible. The 'Body' tab is selected, showing a 'raw' JSON request. The request status is '201 Created' with a response time of 3.23 s and a size of 426 B. The JSON body is as follows:

```
1 {
2   "name": "User2",
3   "email": "user2@test.com",
4   "password": "123456"
5 }
```

Below the request, the 'Body' tab shows the 'JSON' response. The response status is '201 Created' with a response time of 3.23 s and a size of 426 B. The JSON body is as follows:

```
1 {
2   "success": true,
3   "message": "Usuari registrat correctament",
4   "data": {
5     "name": "User2",
6     "email": "user2@test.com",
7     "updated_at": "2026-05-06T09:23:40.000000Z",
8     "created_at": "2026-05-06T09:23:40.000000Z",
9     "id": 2
10  }
11 }
```

Seguim:

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/api/login`. The request body is a JSON object: `{ "email": "user2@test.com", "password": "123456" }`. The response is a 200 OK status with a response time of 1.75 s and a body size of 325 B. The response body is a JSON object: `{ "success": true, "message": "Login correcte", "token": "2|CqhUS9gC5Ky3C6NwwavNSOCfBjsXeoDDv0PcHqXE732553dd" }`.

```
POST http://127.0.0.1:8000/api/login
{
  "email": "user2@test.com",
  "password": "123456"
}
```

200 OK • 1.75 s • 325 B • [Globe Icon] • [More Icon]

```
{
  "success": true,
  "message": "Login correcte",
  "token": "2|CqhUS9gC5Ky3C6NwwavNSOCfBjsXeoDDv0PcHqXE732553dd"
}
```

User2 intenta veure tasca del User1 (He posat el Bearer del U2)

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8000/api/tasks/1`. The response is a 404 Not Found status with a response time of 563 ms and a body size of 274 B. The response body is a JSON object: `{ "success": false, "message": "Tasca no trobada" }`.

```
GET http://127.0.0.1:8000/api/tasks/1
```

404 Not Found • 563 ms • 274 B • [Globe Icon] • [More Icon]

```
{
  "success": false,
  "message": "Tasca no trobada"
}
```

Això és exactament el que ha d'apareixer.

Ara amb el Bearer del User1:

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://127.0.0.1:8000/api/tasks/1
- Body (raw):**

```
1 {
2   "nom": "Estudiar Laravel (actualitzat)",
3   "comentari": "Ara estic provant el UPDATE",
4   "categoria": "Proves"
5 }
```
- Status:** 200 OK
- Response Body (JSON):**

```
1 {
2   "success": true,
3   "message": "Tasca actualitzada",
4   "data": {
5     "id": 1,
6     "user_id": 1,
7     "nom": "Estudiar Laravel (actualitzat)",
8     "comentari": "Ara estic provant el UPDATE",
9     "categoria": "Proves",
10    "created_at": "2026-05-06T09:07:35.000000Z",
11    "updated_at": "2026-05-06T09:39:59.000000Z"
12  }
13 }
```

Ara amb el Bearer del User2:

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://127.0.0.1:8000/api/tasks/1
- Body (raw):**

```
1 {
2   "nom": "Intent il·legal",
3   "comentari": "User2 intenta modificar",
4   "categoria": "Hack"
5 }
```
- Status:** 403 Forbidden
- Response Body (JSON):**

```
1 {
2   "success": false,
3   "message": "No tens permisos"
4 }
```

I això és exactament el que ha de passar amb la seguretat :P

El últim que toca comprovar és el DELETE.

Amb el Bearer del User1:

The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** http://127.0.0.1:8000/api/tasks/1
- Buttons:** Docs, Params, Authorization, Headers 9, Body, Scripts, Tests, Settings, Cookies
- Body Tab:** raw (selected), JSON (dropdown), Schema, Beautify
- Response:** 200 OK, 454 ms, 265 B
- JSON Response:**

```
1 {  
2   "success": true,  
3   "message": "Tasca eliminada"  
4 }
```

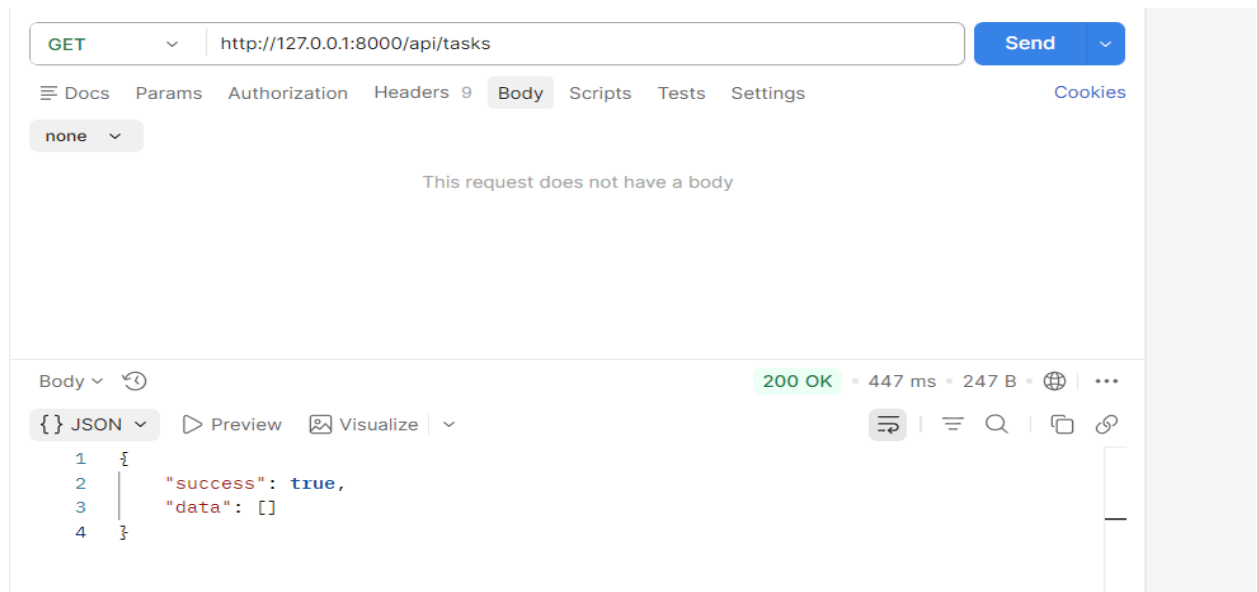
Ara anem a fer la comprovació:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/tasks/1
- Buttons:** Docs, Params, Authorization, Headers 9, Body, Scripts, Tests, Settings, Cookies
- Body Tab:** none (selected)
- Response:** 404 Not Found, 399 ms, 274 B
- JSON Response:**

```
1 {  
2   "success": false,  
3   "message": "Tasca no trobada"  
4 }
```

I per acabar d'assegurar-me..



DELETE OK ✓

Amb totes aquestes passes, ja he complert amb els requisits mínims de l'exercici per assolir el NIVELL 1.

Com m'agraden els reptes i les coses ben fetes, ja saps que sempre intentaré apostar pel millor, així que vaig amb el NIVELL 2.

I després al 3 :P

CATEGORIES

- Crear model i migració de Category

```
C:\Users\HP\api-tasques>php artisan make:model Category -m

INFO Model [C:\Users\HP\api-tasques\app\Models\Category.php] created successfully.

INFO Migration [C:\Users\HP\api-tasques\database\migrations\2026_05_06_100134_create_categories_table.php] created successfully.
```

- Editar migració categories

```
public function up(): void
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();

        // Nom de la categoria.
        $table->string('nom');

        $table->timestamps();
    });
}
```

- Modificar migració tasks -> category_id

```
public function up(): void
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();

        // Cada tasca pertany a un usuari concret.
        $table->foreignId('user_id')->constrained()->onDelete('cascade');

        // Dades principals de la tasca.
        $table->string('nom');

        $table->text('comentari');

        // Relació amb la taula categories.
        $table->foreignId('category_id')->constrained()->onDelete('cascade');

        $table->timestamps();
    });
}
```


- Ara:
php artisan migrate:fresh

```
C:\Users\HP\api-tasques>php artisan migrate:fresh

Dropping all tables ..... 97.45ms DONE

[INFO] Preparing database.

Creating migration table ..... 139.91ms DONE

[INFO] Running migrations.

0001_01_01_000000_create_users_table ..... 169.38ms DONE
0001_01_01_000001_create_cache_table ..... 29.35ms DONE
0001_01_01_000002_create_jobs_table ..... 75.06ms DONE
2026_05_05_193424_create_tasks_table ..... 52.71ms DONE
2026_05_05_200930_create_personal_access_tokens_table ..... 27.00ms DONE
2026_05_06_100134_create_categories_table ..... 10.33ms DONE
```

- Modifico Models/Category.php

```
api-tasques > app > Models > Category.php > PHP > Category

1  <?php
2
3  namespace App\Models;
4  use App\Models\Task;
5
6  use Illuminate\Database\Eloquent\Model;
7
8  0 references | 0 implementations
9  class Category extends Model
10 {
11     // Camps que es poden omplir
12     0 references
13     protected $fillable = [
14         'nom'
15     ];
16
17     // Una categoria té moltes tasques
18     0 references | 0 overrides
19     public function tasks()
20     {
21         return $this->hasMany(Task::class);
22     }
23 }
```

- Modifico Models/Task

```
use App\Models\User;
use App\Models\Category;

10 references | 0 implementations
class Task extends Model
{
    // Camps que Laravel permet omplir des del controlador.
    0 references
    protected $fillable = [
        'user_id',
        'nom',
        'comentari',
        'category_id'
    ];

    // Cada tasca pertany a un usuari.
    0 references | 0 overrides
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    // Cada tasca pertany a una categoria.
    0 references | 0 overrides
    public function category()
    {
        return $this->belongsTo(Category::class);
    }
}
```

- Modificar TaskController

(Ara hem d'adaptar el controlador perquè ja no usem categoria, sinó category_id.)

```
// Crear una nova tasca
1 reference | 0 overrides
public function store(Request $request)
{
    // Validació
    $request->validate([
        'nom' => 'required',
        'comentari' => 'required',
        'category_id' => 'required|exists:categories,id'
    ]);
}
```

```
// Crear tasca associada a l'usuari
$task = Task::create([
    'user_id' => $user->id,
    'nom' => $request->nom,
    'comentari' => $request->comentari,
    'category_id' => $request->category_id
]);
```

```
$task->update([
    'nom' => $request->nom,
    'comentari' => $request->comentari,
    'category_id' => $request->category_id
]);
```

- Creació de Controller de Categories

php artisan make:controller Api/CategoryController --api

```
C:\Users\HP\api-tasques>php artisan make:controller Api/CategoryController --api  
  
INFO Controller [C:\Users\HP\api-tasques\app\Http\Controllers\Api\CategoryController.php] created successfully.
```

- Ara l'edició:

```
CategoryController.php X  
api-tasques > app > Http > Controllers > Api > CategoryController.php > PHP > CategoryController > store()  
1  <?php  
2  
3  namespace App\Http\Controllers\Api;  
4  
5  use App\Http\Controllers\Controller;  
6  use Illuminate\Http\Request;  
7  use App\Models\Category;  
8  
9  0 references | 0 implementations  
10 class CategoryController extends Controller  
11 {  
12     // GET /api/categories  
13     // Llistar categories  
14     0 references | 0 overrides  
15     public function index()  
16     {  
17         $categories = Category::all();  
18         return response()->json([  
19             'success' => true,  
20             'data' => $categories  
21         ], 200);  
22     }  
23 }
```

```
24 // POST /api/categories  
25 // Crear categoria  
26 0 references | 0 overrides  
27 public function store(Request $request)  
28 {  
29     $request->validate([  
30         'nom' => 'required'  
31     ]);  
32  
33     $category = Category::create([  
34         'nom' => $request->nom  
35     ]);  
36  
37     return response()->json([  
38         'success' => true,  
39         'message' => 'Categoria creada correctament',  
40         'data' => $category  
41     ], 201);  
42 }  
43 }
```

- Afegir rutes de Categories a api.php:

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\TaskController;
use App\Http\Controllers\Api\CategoryController;

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/tasks', [TaskController::class, 'index']);
    Route::post('/tasks', [TaskController::class, 'store']);
    Route::get('/tasks/{id}', [TaskController::class, 'show']);
    Route::put('/tasks/{id}', [TaskController::class, 'update']);
    Route::delete('/tasks/{id}', [TaskController::class, 'destroy']);

    Route::get('/categories', [CategoryController::class, 'index']);
    Route::post('/categories', [CategoryController::class, 'store']);
});
```

El pas següent és fer la comprovació amb POSTMAN:

The screenshot shows the Postman interface. At the top, the request method is set to 'POST' and the URL is 'http://127.0.0.1:8000/api/register'. The 'Body' tab is selected, and the request body is in JSON format, containing the following data:

```
{
  "name": "Rut",
  "email": "rut@test.com",
  "password": "123456"
}
```

Below the request body, the response is displayed. The status is '201 Created', the time taken is '927 ms', and the size is '422 B'. The response body is in JSON format, containing the following data:

```
{
  "success": true,
  "message": "Usuari registrat correctament",
  "data": {
    "name": "Rut",
    "email": "rut@test.com",
    "updated_at": "2026-05-06T10:53:06.000000Z",
    "created_at": "2026-05-06T10:53:06.000000Z",
    "id": 1
  }
}
```

POST http://127.0.0.1:8000/api/login Send

Docs Params Authorization Headers 11 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "email": "rut@test.com",
3   "password": "123456"
4 }
```

Body 200 OK • 798 ms • 325 B

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Login correcte",
4   "token": "1|TR08CABYnblz8kmMfw3ob0s4Cv5o5LkIL9aL6DTCb44754d9"
5 }
```

Ara ho creo:

POST http://127.0.0.1:8000/api/categories Send

Docs Params Authorization Headers 11 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "nom": "Estudis"
3 }
```

Body 201 Created • 392 ms • 402 B

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Categoria creada correctament",
4   "data": {
5     "nom": "Estudis",
6     "updated_at": "2026-05-06T10:57:02.000000Z",
7     "created_at": "2026-05-06T10:57:02.000000Z",
8     "id": 1
9   }
10 }
```

POST http://127.0.0.1:8000/api/categories Send

Docs Params Authorization Headers 11 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   | "nom": "Feina"
3   }
```

Body 201 Created • 425 ms • 400 B • ...

{ } JSON Preview Visualize

```
1 {
2   | "success": true,
3   | "message": "Categoria creada correctament",
4   | "data": {
5   |   | "nom": "Feina",
6   |   | "updated_at": "2026-05-06T10:58:02.000000Z",
7   |   | "created_at": "2026-05-06T10:58:02.000000Z",
8   |   | "id": 2
9   | }
10 }
```

Fem un GET per veure les que he creat:

GET http://127.0.0.1:8000/api/categories Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1
```

Body 200 OK • 731 ms • 578 B • ...

{ } JSON Preview Visualize

```
1 {
2   | "success": true,
3   | "data": [
4   |   | {
5   |   |   | "id": 1,
6   |   |   | "nom": "Estudis",
7   |   |   | "created_at": "2026-05-06T10:57:02.000000Z",
8   |   |   | "updated_at": "2026-05-06T10:57:02.000000Z"
9   |   | },
10  |   | {
11  |   |   | "id": 2,
12  |   |   | "nom": "Feina",
13  |   |   | "created_at": "2026-05-06T10:58:02.000000Z",
14  |   |   | "updated_at": "2026-05-06T10:58:02.000000Z"
15  |   | },
16  |   | {
17  |   |   | "id": 3,
18  |   |   | "nom": "Personal",
19  |   |   | "created_at": "2026-05-06T10:58:32.000000Z",
```

Abans de res, faré el DELETE:

```
CategoryController.php X
api-tasques > app > Http > Controllers > Api > CategoryController.php > PHP > CategoryController > destroy()
9  class CategoryController extends Controller
25  public function store(Request $request)
40  }
41
42  // DELETE /api/categories/{id}
0 references | 0 overrides
43  public function destroy($id)
44  {
45      $category = Category::find($id);
46
47      if (!$category) {
48          return response()->json([
49              'success' => false,
50              'message' => 'Categoria no trobada'
51          ], 404);
52      }
53
54      $category->delete();
55
56      return response()->json([
57          'success' => true,
58          'message' => 'Categoria eliminada'
59      ], 200);
60  }
61  }
```

Ara modifico Routes:

```
Route::delete('/categories/{id}', [CategoryController::class, 'destroy']);
```

I ara...

DELETE http://127.0.0.1:8000/api/categories/4

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cook

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

1

Body Cookies Headers 7 Test Results 200 OK • 530 ms • 269 B •

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Categoria eliminada"
4 }
```

Ho comprovo amb GET /Categories:

```
{
  [
    {
      "id": 2,
      "nom": "Feina",
      "created_at": "2026-05-06T10:58:02.000000Z",
      "updated_at": "2026-05-06T10:58:02.000000Z"
    },
    {
      "id": 3,
      "nom": "Personal",
      "created_at": "2026-05-06T10:58:32.000000Z",
      "updated_at": "2026-05-06T10:58:32.000000Z"
    }
  ]
}
```

Genial, això vol dir que s'ha esborrat realment!!!

DELETE CATEGORIES OK!!

Crear TASCA amb CATEGORIA:

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/api/tasks`. The request body is a JSON object: `{ "nom": "Fer exercicis Laravel", "comentari": "Practicar API REST amb categories", "category_id": 1 }`. The response status is `201 Created` with a response time of `1.24 s` and a body size of `488 B`. The response body is a JSON object: `{ "success": true, "message": "Tasca creada correctament", "data": { "user_id": 1, "nom": "Fer exercicis Laravel", "comentari": "Practicar API REST amb categories", "category_id": 1, "updated_at": "2026-05-09T09:30:25.000000Z", "created_at": "2026-05-09T09:30:25.000000Z", "id": 1 } } }`.

Ho he afegit a Categoria 1-> Estudis

Com vull que realment mostri on s'afegeix(i tinc memòria de peix) , vaig a fer un ajust que segur que t'agrada 😊

Modificació a TaskController:

```
class TaskController extends Controller
{
    // GET /api/tasks
    // Llistar totes les tasques de l'usuari autenticat
    1 reference | 0 overrides
    public function index(Request $request)
    {
        // Obtenim l'usuari autenticat a partir del token
        $user = $request->user();

        // Recuperem només les seves tasques
        $tasks = Task::with('category')
            ->where('user_id', $user->id)
            ->get();
    }
}
```

```

public function show(Request $request, $id)
{
    // Obtenim l'usuari autenticat
    $user = $request->user();

    // Busquem la tasca només si pertany a l'usuari autenticat
    $task = Task::with('category')
        ->where('id', $id)
        ->where('user_id', $user->id)
        ->first();
}

```

Ho tenim:

The screenshot shows a REST client interface. At the top, a GET request is configured to `http://127.0.0.1:8000/api/tasks`. The 'Body' tab is selected, showing a 'raw' request body with the number '1'. Below the request, the response is displayed in the 'Body' tab, showing a 200 OK status with a response time of 416 ms and a body size of 569 B. The response is in JSON format and contains the following data:

```

{
  "success": true,
  "data": [
    {
      "id": 1,
      "user_id": 1,
      "nom": "Fer exercicis Laravel",
      "comentari": "Practicar API REST amb categories",
      "category_id": 1,
      "created_at": "2026-05-09T09:30:25.000000Z",
      "updated_at": "2026-05-09T09:30:25.000000Z",
      "category": {
        "id": 1,
        "nom": "Estudis",
        "created_at": "2026-05-06T10:57:02.000000Z",
        "updated_at": "2026-05-06T10:57:02.000000Z"
      }
    }
  ]
}

```

Ara ja surt el nom de la Categoria (a més del seu id) ! :)

Ara afegiré mes tasques per tal d'assignar-les a categories:

POST http://127.0.0.1:8000/api/tasks

more actions Params Authorization Headers 11 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nom": "Repassar autenticació",
3   "comentari": "Comprovar register, login i token",
4   "category_id": 1
5 }
```

Body Cookies Headers 7 Test Results 201 Created 521 ms 615 B

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Tasca creada correctament",
4   "data": {
5     "user_id": 1,
6     "nom": "Repassar autenticació",
7     "comentari": "Comprovar register, login i token",
8     "category_id": 1,
9     "updated_at": "2026-05-09T09:52:15.000000Z",
10    "created_at": "2026-05-09T09:52:15.000000Z",
11    "id": 2,
12    "category": {
13      "id": 1,
14      "nom": "Estudis",
15      "created_at": "2026-05-06T10:57:02.000000Z",
16      "updated_at": "2026-05-06T10:57:02.000000Z"
17    }
18  }
19 }
```

POST http://127.0.0.1:8000/api/tasks

Docs Params Authorization Headers 11 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nom": "Preparar documentació",
3   "comentari": "Fer explicació dels endpoints",
4   "category_id": 2
5 }
```

Body Cookies Headers 7 Test Results 201 Created 423 ms 614 B

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Tasca creada correctament",
4   "data": {
5     "user_id": 1,
6     "nom": "Preparar documentació",
7     "comentari": "Fer explicació dels endpoints",
8     "category_id": 2,
9     "updated_at": "2026-05-09T09:53:36.000000Z",
10    "created_at": "2026-05-09T09:53:36.000000Z",
11    "id": 3,
12    "category": {
13      "id": 2,
14      "nom": "Feina",
15      "created_at": "2026-05-06T10:58:02.000000Z",
16      "updated_at": "2026-05-06T10:58:02.000000Z"
17    }
18  }
19 }
```

POST http://127.0.0.1:8000/api/tasks

Docs Params Authorization Headers 11 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nom": "Organitzar agenda",
3   "comentari": "Revisar tasques pendents",
4   "category_id": 3
5 }
```

Body Cookies Headers 7 Test Results 201 Created 527 ms 598 B

```
1 {
2   "success": true,
3   "message": "Tasca creada correctament",
4   "data": {
5     "user_id": 1,
6     "nom": "Organitzar agenda",
7     "comentari": "Revisar tasques pendents",
8     "category_id": 3,
9     "updated_at": "2026-05-09T09:54:25.000000Z",
10    "created_at": "2026-05-09T09:54:25.000000Z",
11    "id": 4,
12    "category": {
13      "id": 3,
14      "nom": "Personal",
15      "created_at": "2026-05-06T10:58:32.000000Z",
16      "updated_at": "2026-05-06T10:58:32.000000Z"
17    }
18  }
19 }
```

Ara ja tinc tasques per a totes les categories. Comprovo que tot estigui correcte:

GET http://127.0.0.1:8000/api/tasks

Docs Params Authorization Headers 9 Body Scripts Tests Settings

Body Cookies Headers 7 Test Results 200 OK 411 ms 1.5 KB

```
1 {
2   "success": true,
3   "data": [
4     {
5       "id": 1,
6       "user_id": 1,
7       "nom": "Fer exercicis Laravel",
8       "comentari": "Practicar API REST amb categories",
9       "category_id": 1,
10      "created_at": "2026-05-09T09:30:25.000000Z",
11      "updated_at": "2026-05-09T09:30:25.000000Z",
12      "category": {
13        "id": 1,
14        "nom": "Estudis",
15        "created_at": "2026-05-06T10:57:02.000000Z",
16        "updated_at": "2026-05-06T10:57:02.000000Z"
17      }
18    },
19    {
20      "id": 2,
21      "user_id": 1,
22      "nom": "Repasar autenticació",
23      "comentari": "Comprovar register, login i token",
24      "category_id": 1,
25      "created_at": "2026-05-09T09:52:15.000000Z",
26      "updated_at": "2026-05-09T09:52:15.000000Z",
27      "category": {
28        "id": 1,
29        "nom": "Estudis",
30        "created_at": "2026-05-06T10:57:02.000000Z",

```

CONSEGUIRÉ EL 10?! ARA VE L'EXTRA:

Filtrar per categoria → GET /api/tasks?categoria=ID

app/Http/Controllers/Api/TaskController.php

```
class TaskController extends Controller
{
    // GET /api/tasks
    // llistar les tasques de l'usuari autenticat amb filtre opcional per categoria
    1 reference | 0 overrides
    public function index(Request $request)
    {
        // Obtenim l'usuari autenticat a partir del token
        $user = $request->user();

        // Preparem la consulta base: només tasques de l'usuari autenticat
        $query = Task::with('category')
            ->where('user_id', $user->id);

        // Si arriba ?categoria=ID, filtrem per categoria
        if ($request->has('categoria')) {
            $query->where('category_id', $request->categoria);
        }

        // Executem la consulta
        $tasks = $query->get();

        return response()->json([
            'success' => true,
            'data' => $tasks
        ], 200);
    }
}
```

A veure que surt:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/tasks?categoria=1
- Buttons:** Docs, Params, Authorization, Headers, Body (selected), Scripts, Tests, Settings
- Body Tab:** Shows the raw response body as JSON.
- Status:** 200 OK, 9.61 s, 897 B
- JSON Body:**

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "user_id": 1,
      "nom": "Fer exercicis Laravel",
      "comentari": "Practicar API REST amb categories",
      "category_id": 1,
      "created_at": "2026-05-09T09:30:25.000000Z",
      "updated_at": "2026-05-09T09:30:25.000000Z",
      "category": {
        "id": 1,
        "nom": "Estudis",
        "created_at": "2026-05-06T10:57:02.000000Z",
        "updated_at": "2026-05-06T10:57:02.000000Z"
      }
    },
    {
      "id": 2,
      "user_id": 1,
      "nom": "Repasar autenticaci\u00f3",
      "comentari": "Comprovar register, login i token",
      "category_id": 1,
      "created_at": "2026-05-09T09:52:15.000000Z",
      "updated_at": "2026-05-09T09:52:15.000000Z",
      "category": {
        "id": 1,
        "nom": "Estudis",
        "created_at": "2026-05-06T10:57:02.000000Z",
        "updated_at": "2026-05-06T10:57:02.000000Z"
      }
    }
  ]
}
```

Filtrar per categoria → GET /api/tasks?categoria=ID

Ordenació → GET /api/tasks?order=asc

app/Http/Controllers/Api/TaskController.php (index)

```
// Si arriba ?categoria=ID, filtrem per categoria
if ($request->has('categoria')) {
    $query->where('category_id', $request->categoria);
}

// Ordenació opcional
// Per defecte: desc (més noves primer)
$order = $request->query('order', 'desc');

// Només permetem asc o desc
if (!in_array($order, ['asc', 'desc'])) {
    $order = 'desc';
}

// Ordenem per data de creació
$query->orderBy('created_at', $order);

// Executem la consulta
$tasks = $query->get();
```

Primer en ordre ascendent:

GET http://127.0.0.1:8000/api/tasks?order=asc

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

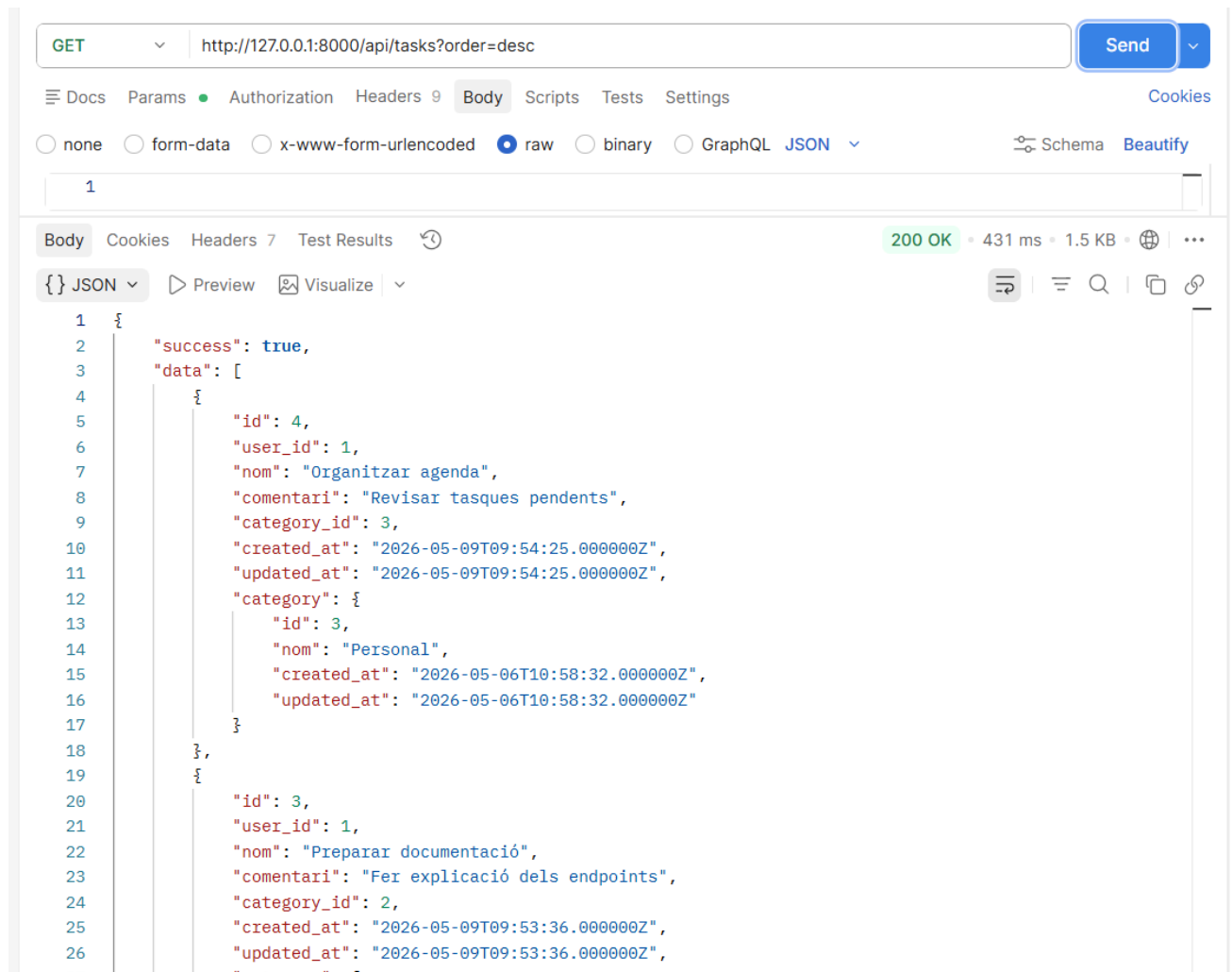
1

Body Cookies Headers 7 Test Results 200 OK • 516 ms • 1.5 KB

{ } JSON Preview Visualize

```
{
  "id": 3,
  "user_id": 1,
  "nom": "Preparar documentació",
  "comentari": "Fer explicació dels endpoints",
  "category_id": 2,
  "created_at": "2026-05-09T09:53:36.000000Z",
  "updated_at": "2026-05-09T09:53:36.000000Z",
  "category": {
    "id": 2,
    "nom": "Feina",
    "created_at": "2026-05-06T10:58:02.000000Z",
    "updated_at": "2026-05-06T10:58:02.000000Z"
  }
},
{
  "id": 4,
  "user_id": 1,
  "nom": "Organitzar agenda",
  "comentari": "Revisar tasques pendents",
  "category_id": 3,
  "created_at": "2026-05-09T09:54:25.000000Z",
  "updated_at": "2026-05-09T09:54:25.000000Z",
  "category": {
    "id": 3,
    "nom": "Personal",
    "created_at": "2026-05-06T10:58:32.000000Z",
    "updated_at": "2026-05-06T10:58:32.000000Z"
  }
}
```

En ordre descendent:



GET Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐ Schema Beautify

1

Body Cookies Headers 7 Test Results 200 OK • 431 ms • 1.5 KB • ...

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "data": [
4     {
5       "id": 4,
6       "user_id": 1,
7       "nom": "Organitzar agenda",
8       "comentari": "Revisar tasques pendents",
9       "category_id": 3,
10      "created_at": "2026-05-09T09:54:25.000000Z",
11      "updated_at": "2026-05-09T09:54:25.000000Z",
12      "category": {
13        "id": 3,
14        "nom": "Personal",
15        "created_at": "2026-05-06T10:58:32.000000Z",
16        "updated_at": "2026-05-06T10:58:32.000000Z"
17      }
18    },
19    {
20      "id": 3,
21      "user_id": 1,
22      "nom": "Preparar documentació",
23      "comentari": "Fer explicació dels endpoints",
24      "category_id": 2,
25      "created_at": "2026-05-09T09:53:36.000000Z",
26      "updated_at": "2026-05-09T09:53:36.000000Z",
27    }
28  ]
29 }
```

Ordenació → GET /api/tasks?order=asc (i desc :P)

Paginació → GET /api/tasks?page=1

app/Http/Controllers/Api/TaskController.php (index)

```
// Executem la consulta i obtenim les tasques paginades (2 per pàgina)
$tasks = $query->paginate(2);
```

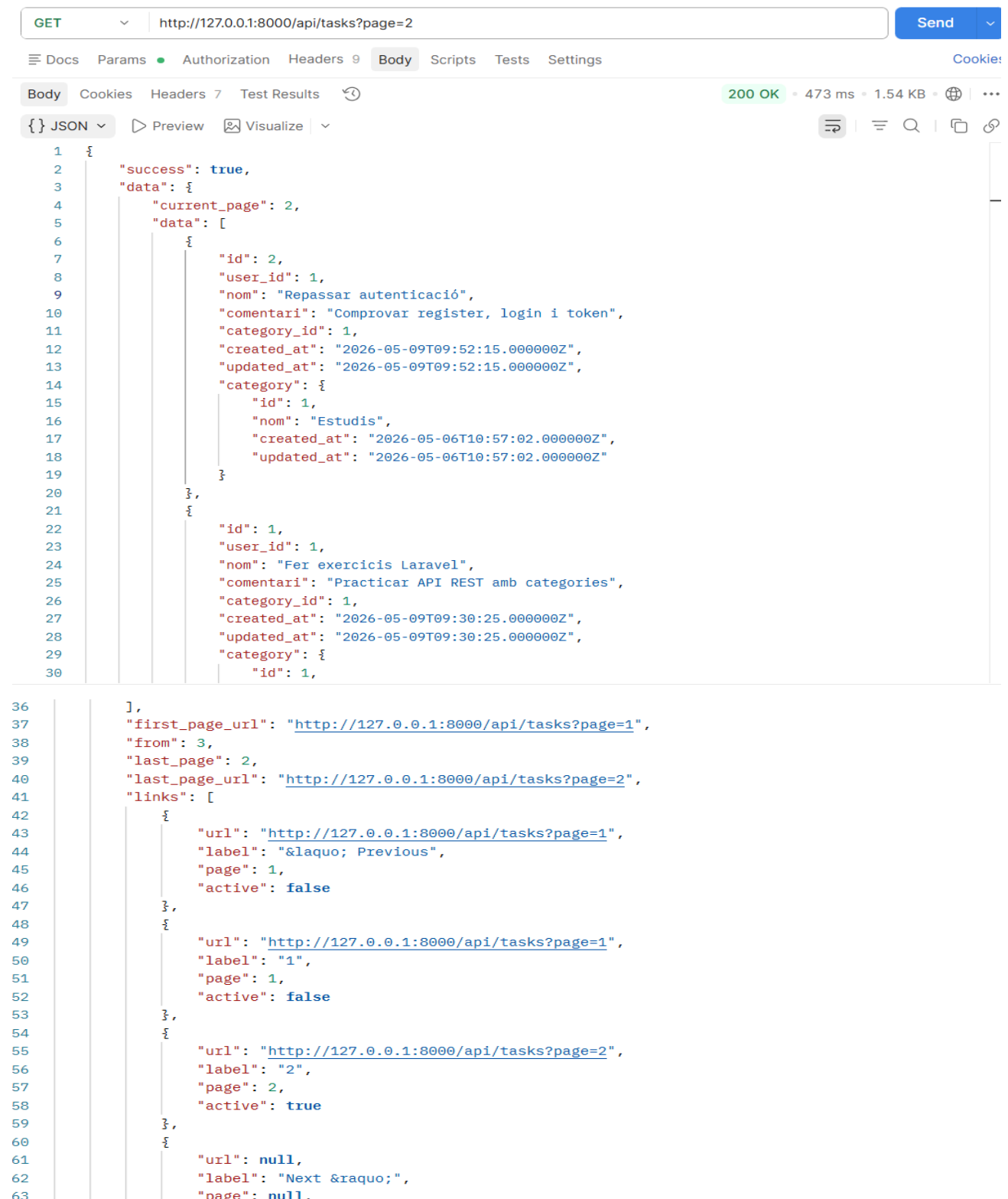
GET http://127.0.0.1:8000/api/tasks?page=1

200 OK • 652 ms • 1.53 KB

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 4,
        "user_id": 1,
        "nom": "Organitzar agenda",
        "comentari": "Revisar tasques pendents",
        "category_id": 3,
        "created_at": "2026-05-09T09:54:25.000000Z",
        "updated_at": "2026-05-09T09:54:25.000000Z",
        "category": {
          "id": 3,
          "nom": "Personal",
          "created_at": "2026-05-06T10:58:32.000000Z",
          "updated_at": "2026-05-06T10:58:32.000000Z"
        }
      },
      {
        "id": 3,
        "user_id": 1,
        "nom": "Preparar documentació",
        "comentari": "Fer explicació dels endpoints",
        "category_id": 2,
        "created_at": "2026-05-06T10:58:32.000000Z",
        "updated_at": "2026-05-06T10:58:32.000000Z",
        "category": {
          "id": 2,
          "nom": "Treball",
          "created_at": "2026-05-06T10:58:32.000000Z",
          "updated_at": "2026-05-06T10:58:32.000000Z"
        }
      }
    ],
    "first_page_url": "http://127.0.0.1:8000/api/tasks?page=1",
    "from": 1,
    "last_page": 2,
    "last_page_url": "http://127.0.0.1:8000/api/tasks?page=2",
    "links": [
      {
        "url": null,
        "label": "&laquo; Previous",
        "page": null,
        "active": false
      },
      {
        "url": "http://127.0.0.1:8000/api/tasks?page=1",
        "label": "1",
        "page": 1,
        "active": true
      },
      {
        "url": "http://127.0.0.1:8000/api/tasks?page=2",
        "label": "2",
        "page": 2,
        "active": false
      }
    ]
  }
}
```


Vaig a provar amb :

GET /api/tasks?page=2



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/tasks?page=2
- Status:** 200 OK
- Response Time:** 473 ms
- Response Size:** 1.54 KB
- Body:** JSON

```
1  {
2    "success": true,
3    "data": {
4      "current_page": 2,
5      "data": [
6        {
7          "id": 2,
8          "user_id": 1,
9          "nom": "Repassar autenticació",
10         "comentari": "Comprovar register, login i token",
11         "category_id": 1,
12         "created_at": "2026-05-09T09:52:15.000000Z",
13         "updated_at": "2026-05-09T09:52:15.000000Z",
14         "category": {
15           "id": 1,
16           "nom": "Estudis",
17           "created_at": "2026-05-06T10:57:02.000000Z",
18           "updated_at": "2026-05-06T10:57:02.000000Z"
19         }
20       },
21       {
22         "id": 1,
23         "user_id": 1,
24         "nom": "Fer exercicis Laravel",
25         "comentari": "Practicar API REST amb categories",
26         "category_id": 1,
27         "created_at": "2026-05-09T09:30:25.000000Z",
28         "updated_at": "2026-05-09T09:30:25.000000Z",
29         "category": {
30           "id": 1,
31           "nom": "Estudis",
32           "created_at": "2026-05-06T10:57:02.000000Z",
33           "updated_at": "2026-05-06T10:57:02.000000Z"
34         }
35       }
36     ],
37     "first_page_url": "http://127.0.0.1:8000/api/tasks?page=1",
38     "from": 3,
39     "last_page": 2,
40     "last_page_url": "http://127.0.0.1:8000/api/tasks?page=2",
41     "links": [
42       {
43         "url": "http://127.0.0.1:8000/api/tasks?page=1",
44         "label": "&laquo; Previous",
45         "page": 1,
46         "active": false
47       },
48       {
49         "url": "http://127.0.0.1:8000/api/tasks?page=1",
50         "label": "1",
51         "page": 1,
52         "active": false
53       },
54       {
55         "url": "http://127.0.0.1:8000/api/tasks?page=2",
56         "label": "2",
57         "page": 2,
58         "active": true
59       },
60       {
61         "url": null,
62         "label": "Next &raquo;",
63         "page": null,
```

Puc donar l'exercici per finalitzat. No em dona temps de fer el refresh Token :(

TAULA ENDPOINTS

Mètode	Endpoint	Protegit amb token	Funció
POST	<i>/api/register</i>	No	Registrar un usuari nou
POST	<i>/api/login</i>	No	Iniciar sessió i obtenir el token
GET	<i>/api/categories</i>	Sí	Llistar categories
POST	<i>/api/categories</i>	Sí	Crear una categoria
DELETE	<i>/api/categories/{id}</i> }	Sí	Eliminar una categoria
GET	<i>/api/tasks</i>	Sí	Llistar les tasques de l'usuari autenticat
POST	<i>/api/tasks</i>	Sí	Crear una tasca
GET	<i>/api/tasks/{id}</i>	Sí	Veure una tasca concreta
PUT	<i>/api/tasks/{id}</i>	Sí	Actualitzar una tasca
DELETE	<i>/api/tasks/{id}</i>	Sí	Eliminar una tasca
GET	<i>/api/tasks?categoria=ID</i>	Sí	Filtrar tasques per categoria
GET	<i>/api/tasks?order=asc</i>	Sí	Tasques de més antiga a més nova
GET	<i>/api/tasks?order=desc</i>	Sí	Tasques de més nova a més antiga
GET	<i>/api/tasks?page=1</i>	Sí	Mostrar tasques paginades

CONCLUSIÓ

Amb aquest projecte he desenvolupat una API REST segura amb Laravel, seguint els requisits indicats a l'enunciat. L'aplicació no utilitza cap interfície web ni vistes Blade, ja que totes les proves s'han fet amb Postman i totes les respostes principals es retornen en format JSON.

Primer he implementat el registre i el login d'usuaris amb Laravel Sanctum, generant un token d'autenticació que després s'envia a les rutes protegides amb el format `Authorization: Bearer TOKEN`. D'aquesta manera, només els usuaris autenticats poden accedir a les funcionalitats principals de l'API.

També he creat un CRUD complet de tasques, permetent llistar, crear, consultar, actualitzar i eliminar tasques. Cada tasca queda associada a l'usuari autenticat mitjançant el camp `user_id`, cosa que garanteix que cada usuari només pugui veure i modificar les seves pròpies dades.

Per millorar el projecte, he afegit categories en una taula pròpia i he relacionat les tasques amb aquestes categories mitjançant `category_id`. A més, he implementat funcionalitats extra com el filtratge per categoria, l'ordenació ascendent o descendent i la paginació de resultats.

Finalment, he comprovat el funcionament de tots els endpoints amb Postman, incloent casos correctes i proves de seguretat, com intentar accedir a una tasca d'un altre usuari. Amb això, el projecte compleix els requisits obligatoris i incorpora millores del nivell excel·lent (o gairebé...).